

Software Requirements Specification

for

Municipal Complaint Management System

Version 1.0 approved

Prepared by:

Aastha Sheoran — 2400290120004

Ananya Rajput — 2400290120042

Aashi Chaudhary — 2400290120003

Aeishikka Tyagi — 2400290120023

Organization: Academic SDLC Project

Date created: 8 September 2026

Table of Contents

- 1. Introduction
 - 1.1 Purpose
 - 1.2 Document Conventions
 - 1.3 Intended Audience and Reading Suggestions
 - 1.4 Product Scope
 - 1.5 References
- 2. Overall Description
 - 2.1 Product Perspective
 - 2.2 Product Functions
 - 2.3 User Classes and Characteristics
 - 2.4 Operating Environment
 - 2.5 Design and Implementation Constraints
 - 2.6 User Documentation
 - 2.7 Assumptions and Dependencies
- 3. External Interface Requirements
 - 3.1 User Interfaces
 - 3.2 Hardware Interfaces
 - 3.3 Software Interfaces
 - 3.4 Communications Interfaces
- 4. System Features
 - 4.1 User Authentication and Role Management
 - 4.2 Complaint Registration and Evidence
 - 4.3 Complaint Verification and Assignment
 - 4.4 Complaint Tracking and Status Management
 - 4.5 Field Worker Resolution
 - 4.6 Notifications and Feedback
 - 4.7 Search, Dashboards and Reporting
 - 4.8 AI-Assisted Complaint Intelligence
 - 4.9 Administration
- 5. Other Nonfunctional Requirements
 - 5.1 Performance Requirements
 - 5.2 Safety Requirements
 - 5.3 Security Requirements
 - 5.4 Software Quality Attributes
 - 5.5 Business Rules
- 6. Other Requirements
- Appendix A: Glossary
- Appendix B: Analysis Models

Appendix C: To Be Determined List

Appendix D: Additional Analysis Models

D.1 Data Flow Diagram (DFD) — Level 0

D.2 Decision Tree — Municipal Complaint Management System

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and nonfunctional requirements for the Municipal Complaint Management System (MCMS), Version 1.0. The product is a web-based platform that enables citizens to submit and track municipal complaints and enables municipal officers, field workers, and administrators to manage the complaint lifecycle.

The SRS covers the complete core complaint-management subsystem, including authentication, complaint registration, categorization, location and evidence capture, verification, department and worker assignment, status tracking, resolution verification, notifications, feedback, reporting, administration, and selected AI-assisted capabilities.

1.2 Document Conventions

The requirements in this document use the word “shall” for mandatory system behavior and “may” for optional or planned behavior. Each functional requirement is uniquely identified using an FR tag; AI-assisted requirements use AI-FR tags; nonfunctional requirements use NFR tags. Requirement priorities are expressed as High, Medium, or Low.

Terms and abbreviations are defined in Appendix A. UML and other analysis models referenced in Appendix B are intended to be maintained consistently with the requirements in this SRS.

1.3 Intended Audience and Reading Suggestions

This document is intended for the student development team, project supervisor, software designers, developers, testers, documentation writers, and evaluators of the Municipal Complaint Management System.

Readers should first review Sections 1 and 2 for purpose, scope, users, and system context. Section 3 describes external interfaces. Section 4 contains detailed system features and functional requirements. Section 5 defines nonfunctional requirements and business rules. Section 6 and the appendices contain supporting requirements, terminology, analysis-model references, and unresolved items.

1.4 Product Scope

MCMS provides a centralized digital mechanism for reporting and managing municipal service complaints. Citizens can report issues such as potholes, garbage accumulation, water leakage, drainage problems, broken streetlights, sanitation issues, park maintenance problems, damaged public property, and environmental issues.

The system improves transparency and accountability by assigning each complaint a unique identifier, maintaining a status history, routing verified complaints to appropriate departments and workers, collecting resolution evidence, allowing officer verification, notifying citizens, and collecting post-resolution feedback.

The core product is independent of AI and remains operational if AI-assisted functions are unavailable. AI is used only as an assistive layer for classification, priority recommendation, and possible duplicate detection.

1.5 References

- Software Requirements Specification template provided for this project, based on the Karl E. Wiegers SRS structure.
- Municipal Complaint Management System Requirements Report, Version 1.0, prepared for the project.
- Project UML/analysis models to be developed as companion artifacts to this SRS.
- Technology documentation for React.js, Node.js, Express.js, MongoDB, JWT, bcrypt, Multer, Cloudinary, Leaflet/OpenStreetMap, Recharts, and Postman, as applicable during implementation.

2. Overall Description

2.1 Product Perspective

MCMS is a new, self-contained web application for centralized municipal complaint management. It follows a client-server architecture in which a React-based frontend communicates with a Node.js/Express backend through REST APIs. MongoDB stores application data. External services may provide image storage and mapping functionality. AI services may provide assistive classification, priority prediction, and duplicate detection.

The high-level interaction is: Citizen/Officer/Worker/Administrator → React Frontend → REST API → Node.js/Express Backend → MongoDB, with optional connections to Cloudinary, mapping services, notification services, and AI services.

2.2 Product Functions

- User registration, authentication, logout, and role-based access control.
- Citizen complaint creation, categorization, location capture, and evidence upload.
- Unique complaint ID generation and complaint history.
- Officer complaint review, verification, rejection, prioritization, and assignment.
- Department and field-worker assignment.
- Worker progress updates and resolution evidence upload.
- Officer resolution verification and complaint reopening.
- Citizen complaint tracking, notifications, rating, and feedback.
- Deadline, overdue, and escalation management.
- Search, filtering, dashboards, analytics, and reports.
- Administrative management of users, departments, categories, and workers.
- AI-assisted complaint classification, priority prediction, and possible duplicate detection.

2.3 User Classes and Characteristics

User Class	Characteristics and Main Functions
Citizen	General public user. Requires a simple interface. Registers/logs in, submits complaints, uploads evidence, tracks status, receives notifications, requests reopening, and gives feedback.
Municipal Officer	Authorized municipal staff. Reviews and verifies complaints, assigns departments/workers, manages priorities and deadlines, monitors progress, verifies resolution, and generates reports.
Field Worker	Operational staff. Views assigned tasks, complaint details and locations, updates progress, uploads completion evidence, and marks assigned work completed.
Administrator	System-level privileged user. Manages users, officers, workers, departments, categories, settings, and overall reports.

2.4 Operating Environment

- Client devices: desktops, laptops, tablets, and smartphones.
- Operating systems: Windows, macOS, Linux, Android, and iOS through supported modern browsers.
- Supported browsers: current versions of Google Chrome, Microsoft Edge, and Mozilla Firefox.
- Frontend runtime: modern JavaScript-enabled browser.
- Backend runtime: Node.js.
- Backend framework: Express.js.
- Database: MongoDB.
- Development tools: VS Code, Git, GitHub, and Postman.

2.5 Design and Implementation Constraints

- The project shall use the agreed web technology stack: React.js, Node.js, Express.js, and MongoDB.
- Authentication shall use JWT-based authorization and bcrypt for password hashing.
- Uploaded evidence shall be validated for supported type and size.
- External map, image-storage, notification, and AI services may impose API limits, availability constraints, or configuration requirements.
- The application shall use role-based access control to prevent unauthorized operations.
- The project is initially an academic prototype and may not include production-grade municipal integrations.
- AI recommendations shall not replace authorized municipal officer decisions.

2.6 User Documentation

- Citizen user guide covering registration, login, complaint submission, tracking, reopening, and feedback.
- Municipal officer guide covering verification, assignment, monitoring, resolution verification, escalation, and reports.
- Field worker guide covering assigned complaints, progress updates, evidence upload, and completion.
- Administrator guide covering user, department, category, and system management.
- Basic help/instruction text within major user-interface screens.

2.7 Assumptions and Dependencies

- Users have access to the internet and a supported browser/device.
- Citizens and municipal staff provide accurate information.
- Municipal departments, officers, and field workers are registered in the system.
- Field workers have a smartphone or computer when progress/evidence must be updated.
- MongoDB and required backend services are available during operation.
- Map and cloud storage services are available when their functionality is used.
- AI services/models are available when AI-assisted functions are enabled.
- AI output is treated as advisory and may require human verification.

3. External Interface Requirements

3.1 User Interfaces

The application shall provide responsive, role-specific interfaces. Common interface elements shall include navigation, clear action controls, validation messages, status indicators, loading indicators, and error messages.

- Citizen interface: registration, login, dashboard, complaint form, complaint details/timeline, notifications, feedback, and profile.
- Officer interface: dashboard, complaint queue, search/filter controls, complaint review, assignment, resolution verification, escalation, and reports.
- Worker interface: assigned complaint list, complaint details, location, progress update, evidence upload, and completion.
- Administrator interface: user management, department management, category management, worker/officer management, system monitoring, and reports.

3.2 Hardware Interfaces

- Smartphones/tablets may be used by citizens and field workers for mobile access and camera-based evidence capture.
- Desktop/laptop devices may be used by all roles.
- No specialized hardware is required for the core application.

3.3 Software Interfaces

Interface	Purpose / Data Exchanged
MongoDB	Stores users, complaints, categories, departments, assignments, evidence metadata, notifications, status history,

	and feedback.
Cloudinary or equivalent storage	Stores uploaded complaint and resolution images; application stores associated URLs/metadata.
Leaflet + OpenStreetMap	Displays and selects complaint locations.
JWT/bcrypt libraries	Authentication, authorization tokens, and secure password hashing.
AI service/model	Optional text analysis for classification, priority recommendation, and duplicate detection.
Recharts	Client-side visualization of complaint and performance statistics.
Postman	Development-time API testing; not a production dependency.

3.4 Communications Interfaces

Frontend-backend communication shall use HTTP/HTTPS-based REST APIs with structured JSON requests and responses. HTTPS should be used for deployed environments to protect credentials and application data in transit.

- REST API communication between React frontend and Express backend.
- JSON for standard API payloads.
- Multipart/form-data for supported evidence uploads.
- HTTPS for secure production communication.
- Optional email/SMS/push communication may be integrated in later versions.

4. System Features

4.1 User Authentication and Role Management

Priority: High

Stimulus/Response Sequences

1. User opens registration/login → system displays form.
2. User submits valid credentials → system authenticates and creates a session/token.
3. System identifies role → user is redirected to the appropriate dashboard.
4. Unauthorized user requests restricted resource → system denies access.

Functional Requirements

- FR-01: The system shall allow citizens to register using name, email, phone number, address, and password.
- FR-02: The system shall authenticate registered users using valid credentials.
- FR-03: The system shall support Citizen, Municipal Officer, Field Worker, and Administrator roles.
- FR-04: The system shall enforce role-based access control on protected functions.
- FR-05: Passwords shall be stored using secure hashing rather than plaintext.
- FR-06: Invalid credentials shall produce an appropriate error message without revealing sensitive information.

4.2 Complaint Registration and Evidence

Priority: High

Stimulus/Response Sequences

5. Citizen selects Submit Complaint → system displays complaint form.
6. Citizen enters title, description, category, location, and evidence → system validates input.
7. Valid submission → system stores complaint and generates a unique complaint ID.
8. Invalid submission → system identifies missing/invalid fields and requests correction.

Functional Requirements

- FR-07: The system shall allow citizens to submit municipal complaints.
- FR-08: A complaint shall include title, description, category, location, and creation timestamp.
- FR-09: The system shall support complaint categories including Roads, Garbage, Water Supply, Drainage, Electricity/Streetlights, Sanitation, Parks, Public Property, Environment, and Other.
- FR-10: The system shall allow citizens to upload supported photographic evidence.
- FR-11: The system shall validate uploaded file type and size.
- FR-12: The system shall generate a unique complaint ID for each accepted complaint.
- FR-13: The system shall record complaint creation and update timestamps.

4.3 Complaint Verification and Assignment

Priority: High

Stimulus/Response Sequences

9. Officer opens pending complaint → system displays complaint details and evidence.
10. Officer verifies complaint → system changes status to Verified.
11. Officer rejects complaint → system records rejection reason and changes status to Rejected.
12. Verified complaint → officer selects department and worker → system creates assignment and notifies worker.

Functional Requirements

- FR-14: Officers shall be able to view submitted complaints.
- FR-15: Officers shall be able to verify or reject complaints.
- FR-16: Rejection shall require a reason.
- FR-17: Officers shall be able to modify category and priority after review.
- FR-18: Verified complaints shall be assignable to an appropriate municipal department.
- FR-19: Officers shall be able to assign a complaint to a field worker.
- FR-20: Assignment shall record complaint, worker, department, assigning officer, assignment date, priority, and deadline.

4.4 Complaint Tracking and Status Management

Priority: High

Stimulus/Response Sequences

13. Citizen opens My Complaints → system lists the citizen's complaints.
14. Citizen selects complaint → system displays current status and status history.
15. Authorized staff changes status → system records the change and timestamp.
16. Deadline passes before resolution → system flags complaint as overdue.

Functional Requirements

- FR-21: The system shall maintain complaint statuses including Submitted, Under Review, Verified, Assigned, In Progress, Resolved, Closed, Rejected, Reopened, and Overdue.
- FR-22: Citizens shall be able to view the status of their own complaints.
- FR-23: The system shall maintain status history including previous status, new status, actor, and timestamp.
- FR-24: The system shall support complaint deadlines based on configured priority/category rules.
- FR-25: The system shall identify complaints whose deadlines have passed.
- FR-26: Authorized officers shall be able to escalate overdue complaints.

4.5 Field Worker Resolution

Priority: High

Stimulus/Response Sequences

17. Worker opens assigned task → system displays complaint details and location.
18. Worker starts work → system records progress.
19. Worker uploads completion evidence → system stores evidence.
20. Worker marks task completed → system sends it for officer resolution verification.

Functional Requirements

- FR-27: Workers shall view only complaints assigned to them.
- FR-28: Workers shall view complaint description, category, location, priority, and deadline.
- FR-29: Workers shall update progress for assigned complaints.
- FR-30: Workers shall upload resolution evidence.
- FR-31: The system shall allow before/after evidence when applicable.
- FR-32: Workers shall mark work as completed only for assigned complaints.
- FR-33: Officers shall be able to approve or reject submitted resolution evidence.
- FR-34: Rejected resolution shall allow the complaint to return to an active/reopened state.

4.6 Notifications and Feedback

Priority: Medium

Stimulus/Response Sequences

21. Complaint event occurs → system creates a notification for the relevant user.
22. Citizen opens notification → system displays event details.
23. Complaint reaches resolved/closed state → system makes feedback available to the citizen.

Functional Requirements

- FR-35: The system shall notify users about important complaint events.
- FR-36: Notifications shall include complaint ID and relevant status/event information where applicable.
- FR-37: Citizens shall be able to view their notifications.
- FR-38: Citizens shall be able to provide a 1–5 rating after resolution.
- FR-39: Citizens shall be able to submit optional comments with feedback.
- FR-40: Citizens shall be able to request reopening of a resolved complaint; an authorized officer shall review the request.

4.7 Search, Dashboards and Reporting

Priority: Medium

Stimulus/Response Sequences

24. Officer/admin selects filters → system queries matching complaints.
25. Dashboard loads → system calculates summary statistics.
26. User requests report → system presents the requested aggregated information.

Functional Requirements

- FR-41: Officers and administrators shall search complaints by complaint ID.
- FR-42: Complaints shall be filterable by category, status, priority, department, worker, location, and date.
- FR-43: Citizen dashboards shall show complaint counts and recent complaint activity.
- FR-44: Officer dashboards shall show total, pending, in-progress, resolved, overdue, and high-priority complaints.
- FR-45: Worker dashboards shall show assigned, pending, in-progress, and completed work.
- FR-46: Administrators shall be able to view system-level complaint statistics.
- FR-47: The system shall provide reports based on category, department, location, status, priority, date, resolution time, worker performance, and feedback.

4.8 AI-Assisted Complaint Intelligence

Priority: Medium

Stimulus/Response Sequences

27. Citizen submits complaint text → AI module analyzes text → system suggests category/subcategory.
28. Complaint information is analyzed → system suggests priority.
29. New complaint is submitted → system compares it with existing complaints → system flags possible duplicates when similarity is sufficiently high.
30. Officer reviews AI suggestions → officer accepts or modifies them.

Functional Requirements

- AI-FR-01: The system may analyze complaint text and suggest a category/subcategory.
- AI-FR-02: The system may analyze complaint information and suggest Critical, High, Medium, or Low priority.
- AI-FR-03: The system may compare new complaints with existing complaints and identify potentially similar complaints.
- AI-FR-04: AI recommendations shall be advisory and shall not override authorized officer decisions.
- AI-FR-05: If AI services are unavailable, the core complaint-management functions shall continue to operate using manual selection and review.

4.9 Administration

Priority: High

Stimulus/Response Sequences

31. Administrator opens management module → system verifies administrator role.
32. Administrator adds/updates/removes a record → system validates and stores the change.
33. Administrator views system statistics → system displays aggregated information.

Functional Requirements

- FR-48: Administrators shall manage citizen/officer/worker accounts subject to system policy.
- FR-49: Administrators shall manage municipal departments.
- FR-50: Administrators shall manage complaint categories and subcategories.
- FR-51: Administrators shall manage field worker and officer assignments to departments.
- FR-52: Administrators shall manage configurable complaint/deadline rules.
- FR-53: Administrative actions shall be restricted to authorized administrators.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- NFR-01: Under normal prototype load, common user requests should receive a response within approximately 3 seconds.
- NFR-02: Complaint list and dashboard queries should use appropriate database indexing and pagination where necessary.
- NFR-03: File uploads shall provide progress/error feedback and shall not block unrelated application functions.
- NFR-04: AI-assisted operations should return a usable result within a reasonable application timeout; failure shall fall back to manual processing.

5.2 Safety Requirements

- NFR-05: The system shall not be used as a substitute for emergency services. Safety-critical complaints should display appropriate guidance to contact relevant emergency authorities when necessary.
- NFR-06: AI-generated priority classifications shall not be treated as guaranteed safety assessments.

- NFR-07: The system shall prevent accidental deletion or irreversible modification of complaint records through normal user workflows.
- NFR-08: Resolution evidence shall be retained according to the application's data-retention policy.

5.3 Security Requirements

- NFR-09: The system shall authenticate users before protected operations.
- NFR-10: Role-based authorization shall be enforced on the server side.
- NFR-11: Passwords shall be securely hashed using bcrypt or an equivalent secure password-hashing mechanism.
- NFR-12: JWT tokens shall be validated for protected API requests.
- NFR-13: The system shall validate and sanitize user inputs.
- NFR-14: File uploads shall be restricted by allowed types and sizes.
- NFR-15: Production communication should use HTTPS.
- NFR-16: Users shall not be able to access complaints or administrative data outside their authorized scope.
- NFR-17: Sensitive authentication information shall not be exposed in error messages or logs.

5.4 Software Quality Attributes

Attribute	Requirement
Usability	The interface should be simple enough for a first-time citizen to submit a complaint without specialized training.
Reliability	Complaint data and status history should remain consistent during normal operation.
Availability	The deployed system should be available continuously except planned maintenance.
Maintainability	The application should use modular frontend, backend, database, and service components.
Scalability	The architecture should support growth in users, complaints, departments, and evidence records.
Portability	The web application should work across common modern browsers and device classes.
Testability	Core services and APIs should be independently testable using unit and integration tests.
Interoperability	REST/JSON interfaces should allow future integration with external municipal systems.
Correctness	Complaint state transitions and role permissions shall conform to the business rules.

5.5 Business Rules

- BR-01: A citizen must be authenticated before submitting a standard complaint.
- BR-02: Every accepted complaint must have a unique complaint ID.
- BR-03: Every complaint must have a valid category.
- BR-04: Only authorized officers may verify/reject complaints.
- BR-05: Only authorized officers/admins may assign complaints to workers.
- BR-06: A worker may update only complaints assigned to that worker.
- BR-07: Resolution evidence should be provided before work is submitted for final verification.
- BR-08: Only an authorized officer may verify the final resolution.
- BR-09: Citizens may submit feedback after a complaint is resolved.
- BR-10: Citizens may request reopening after resolution, subject to officer review.
- BR-11: AI recommendations are advisory; final municipal decisions remain with authorized staff.
- BR-12: Users cannot access functions outside their assigned role.

6. Other Requirements

6.1 Database Requirements

- The database shall store users, complaints, departments, categories, workers, assignments, evidence metadata, notifications, status history, and feedback.
- Complaint records shall reference the citizen who submitted them.
- Assignment records shall reference the complaint, worker, department, and assigning officer.
- Status history shall preserve chronological complaint-state changes.
- Evidence metadata shall include uploader, complaint reference, file type/URL, and upload timestamp.
- Appropriate indexes should be created for complaint ID, status, category, department, priority, worker, and date fields.

6.2 Internationalization

The initial release shall support English. Multilingual/Hindi and regional-language complaint processing is identified as a future enhancement. AI multilingual processing may be added in a later release.

6.3 Data Retention

Complaint records, status history, feedback, and evidence should be retained according to the project's defined data-retention policy. Exact municipal/legal retention periods are TBD for the academic prototype.

Appendix A: Glossary

Term	Definition
MCMS	Municipal Complaint Management System.
Citizen	Public user who submits and tracks municipal complaints.
Municipal Officer	Authorized municipal staff member who verifies, assigns, monitors, and validates complaints.
Field Worker	Municipal worker responsible for carrying out assigned physical work.
Administrator	User with system-level management privileges.
Complaint ID	Unique identifier assigned to each complaint.
Department	Municipal organizational unit responsible for a category of civic service.
SLA	Service Level Agreement; target time within which a complaint should be addressed/resolved.
JWT	JSON Web Token used for authenticated API access.
AI	Artificial Intelligence.
NLP	Natural Language Processing; analysis of human language text.
REST API	HTTP-based application programming interface used by frontend and backend.
Evidence	Photo or other supporting material attached to a complaint or resolution.
Escalation	Process of forwarding an overdue or unresolved complaint to a higher authority.
Reopen	Changing a resolved complaint back to an active state because the issue remains unresolved.

Appendix B: Analysis Models

The following analysis models are planned as companion artifacts and shall be kept consistent with this SRS:

- Use Case Diagram — actors and major system interactions.
- Class Diagram — major domain classes and relationships.

- Activity Diagram — end-to-end complaint lifecycle.
- Sequence Diagram — complaint submission, assignment, and resolution interactions.
- State Diagram — complaint status transitions.
- Entity-Relationship Diagram — database entities and relationships.
- Component Diagram — frontend, backend, database, storage, maps, notification, and AI components.
- Deployment Diagram — deployment nodes and communication paths.

Appendix C: To Be Determined List

TBD ID	Item	Status / Resolution Target
TBD-01	Exact municipal SLA values by category and priority	To be finalized during implementation/testing.
TBD-02	Exact complaint evidence file-size and format limits	To be finalized based on selected storage configuration.
TBD-03	Production notification provider (email/SMS/push)	Not required for core prototype; future decision.
TBD-04	Exact AI model/service and training dataset	To be selected during AI module implementation.
TBD-05	Exact legal/data-retention requirements for a real municipal deployment	Outside academic prototype scope; requires municipal policy/legal review.
TBD-06	Production hosting and deployment configuration	To be finalized before deployment.

Appendix D: Additional Analysis Models

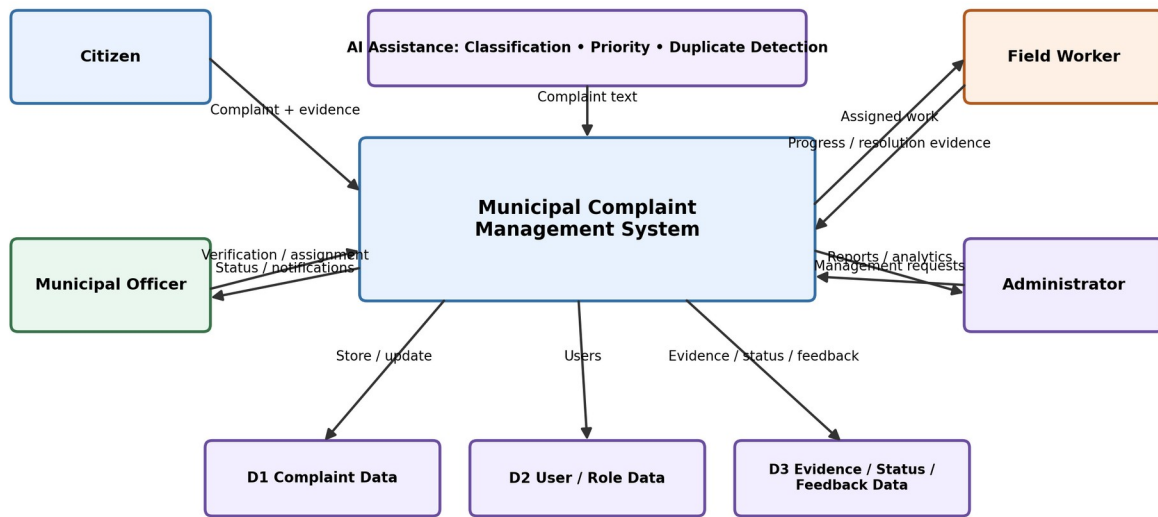
To complement the UML analysis models, this SRS includes a Level 0 Data Flow Diagram (DFD) and a complete decision tree for the Municipal Complaint Management System. These models provide an additional view of system-wide data movement and the major decisions across the complaint lifecycle.

D.1 Data Flow Diagram (DFD) — Level 0

The Level 0 DFD represents the complete project at a high level. It shows the main external actors—Citizen, Municipal Officer, Field Worker, and Administrator—the central Municipal Complaint Management System, the AI assistance layer, and the principal data stores. It covers complaint submission and evidence, verification and assignment, worker progress and resolution evidence, administration and reporting, notifications, and persistent data.

Figure D.1 — Level 0 Data Flow Diagram

DFD Level 0 — Municipal Complaint Management System



External services such as maps, image storage and notifications may connect through the backend.

D.2 Decision Tree — Municipal Complaint Management System

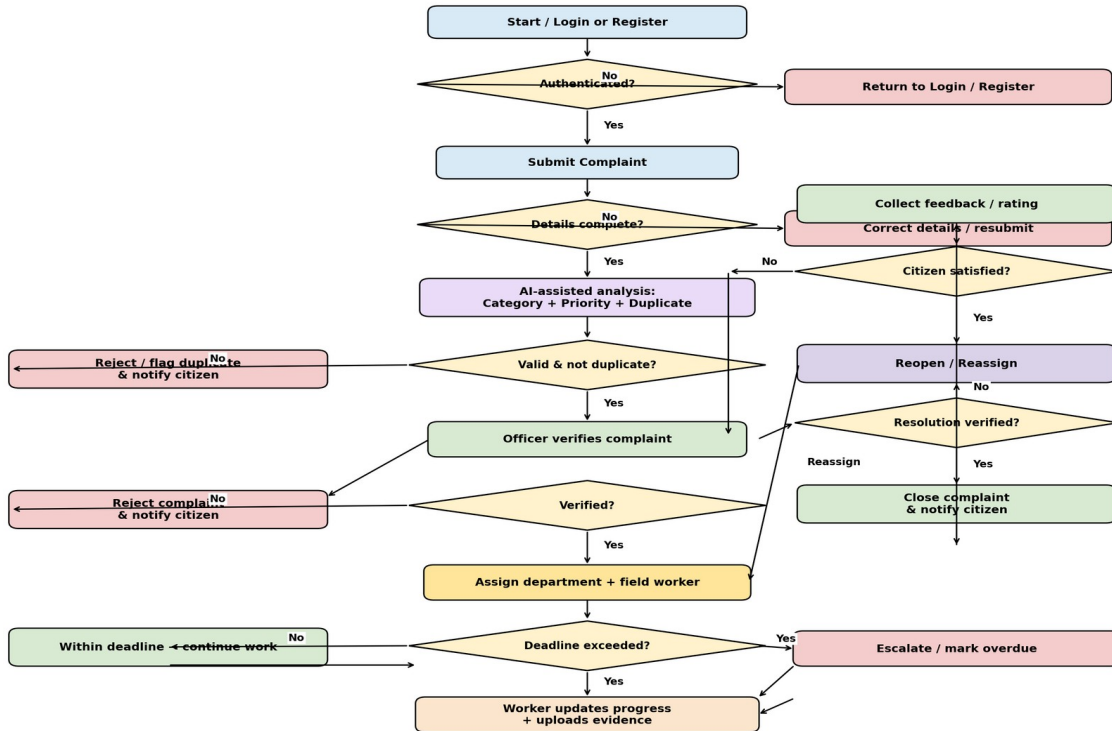
The decision tree represents the complete system workflow rather than only the AI module. It begins with user access and authentication, continues through complaint submission, validation and duplicate checking, AI-assisted classification and priority recommendation, officer verification, department and field-worker assignment, progress and resolution evidence, deadline and overdue handling, escalation, resolution verification, closure, feedback, and reopening.

AI-assisted outputs are advisory only. Authorized municipal officers retain final decision-making authority, and the core complaint-management workflow remains usable when AI services are unavailable.

Figure D.2 — Complete Municipal Complaint Management System Decision Tree

Complete Municipal Complaint Management System — Decision Tree

All decision points show both Yes and No outcomes; AI is advisory only.



AI recommendations support classification, priority and duplicate detection; municipal officers retain final authority.

The decision tree is an analysis model for understanding the end-to-end workflow; it is not intended to replace detailed functional requirements or define a fixed production algorithm.